

4.1 Next.js, TypeScript and Tailwind

- Now we have started the fourth and final week of this project. We will be creating a front end for this product. Then once

Learning objective

What does Next.js give you over plain React? Routing, server versus client components, the app router

React mainly gives you the tools for building UI components, but Next.js gives you a full application framework around React. One of the biggest additions is **routing**. With the App Router, folders and `page.tsx` files automatically become routes, so you do not have to install and configure a separate routing library. Next.js also supports **Server Components and Client Components**. Server Components can render on the server and are useful when the component does not need browser interaction, while Client Components are used when you need things like state, click handlers or browser APIs. The App Router also gives you layouts and nested routing through the project folder structure.

Why do we use TypeScript? Be specific — what mistakes does it actually catch that plain JavaScript wouldn't?

TypeScript catches mistakes before the application runs by checking the types of values being passed around the program. For example, if a component expects a temperature as a `number` but someone gives it `"7"` as a string, plain JavaScript would allow that and the mistake may only appear later at runtime. TypeScript can immediately flag it in VS Code or during the build. It can also catch missing object properties, incorrect function arguments, wrong return types and component props with the wrong type. This is especially useful as applications get larger because it catches many mistakes while you are writing the code rather than after deployment.

What is Tailwind, and why utility classes on the element instead of separate CSS files? What's the argument against it?

Tailwind is a **utility-first CSS framework**. Instead of creating a separate CSS class such as `.card` and then writing all its styles in another file, you place small utility classes directly on the element, for example `p-4`, `rounded-lg` or `font-bold`. This can make development faster because you can see most of a component's styling in the same file as the HTML/JSX and you do not have to constantly switch between component and CSS files. The downside is that elements can end up with very long `className` values, which some developers find harder to read and more cluttered than traditional CSS. Next.js currently supports Tailwind directly in its standard project setup.

Task



- Create a Next.js app with TypeScript and Tailwind
- Two pages, with navigation between them
- One component that takes typed props — define the type properly rather than reaching for `any`

For this task I have create a new repo too keep the frontend and backend complete separate from each other. The repo is called 4week-project-frontend, and for this ticket we will be doing Create Next.js app → Use TypeScript → Use Tailwind → Make 2 pages → Add navigation → Create 1 component with typed props → Cause a real TypeScript error → Fix it.

The first thing that I did was clone the repo to my device and create the new files we would be working form.

Then I created a next.js folder in the 4week-project-front end folder. I used this command to make it `npx create-next-app@latest .` This is because the “.” Represents the current folder I am working in if I just did `npx create-next-app@latest` then it would create it else where in the device.

Then the task specifically says I need to create two pages with a navigation bar between them. therefore I believe I should make the GUI like this

Page one → Home page

Page two → Reading page / view readings

Development

I first went into the app folder that the next.js command had created and changed the `page.tsx` to contain the following code. (This file will be our home page)

```

app > readings > page.tsx > ReadingsPage
1  import Link from "next/link";
2
3  export default function ReadingsPage() {
4    return (
5      <main className="min-h-screen p-10">
6        <h1 className="text-4xl font-bold">
7          Temperature Readings
8        </h1>
9
10       <p className="mt-4 text-lg">
11         Cold-chain readings will be displayed here.
12       </p>
13
14       <nav className="mt-8">
15         <Link
16           href="/"
17           className="rounded bg-blue-600 px-4 py-2 text-white"
18         >
19           Back to Home
20         </Link>
21       </nav>
22     </main>
23   );
24 }

```

The important part of this code is `[import Link from "next/link";]` Next.js gives us its own `Link` component for navigating between pages. (I used this to connect the two pages).

Then I created a new file in the app readings and create a page.tsx in that page with the exact same code. the only difference being it linked back to the home page and had different text in the headers and labels.

Then I need to create a component file that makes the typed props.

```

8  export default function ReadingCard({
9    branchId,
10   minTempC,
11   maxTempC,
12   status,
13 }: ReadingCardProps) {
14   return (
15     <div className="mt-6 rounded-lg border p-6">
16       <h2 className="text-xl font-bold">
17         {branchId}
18       </h2>
19
20       <p className="mt-2">
21         Minimum Temperature: {minTempC}°C
22       </p>
23
24       <p>
25         Maximum Temperature: {maxTempC}°C
26       </p>
27
28       <p className="mt-2 font-semibold">
29         Status: {status}
30       </p>
31     </div>
32   );
33 }

```

The important part of this code is:

```
1 type ReadingCardProps = {  
2   branchId: string;  
3   minTempC: number;  
4   maxTempC: number;  
5   status: string;  
6 };
```

is TypeScript.

We are saying:

branchId → MUST be a string

minTempC → MUST be a number

maxTempC → MUST be a number

status → MUST be a string

Then here:

```
1 }: ReadingCardProps)
```

we tell TypeScript:

These are the exact types of props this component accepts.

We're deliberately **not** doing:

```
1 props: any
```

Because that would throw away most of the protection TypeScript gives us and give us errors.

NOTE: React is compatible / can use typescript wth in the same file as other code types for example; I can be using react file and have typescript and tailwind code in the same folder.

NOTE: A prop gives a value to a value. for example if you have a header called brachId in your tailwind code, it will throw an error because its type is any. A prop defines wherever that header will be a string, number or boolean.

There are three types of props:

Defining props - what we just talked about

Optional props - for this you put ? in front of the defining . It is used for things like extra text o the values underneath a header.

```

6 export default function App({ headerText, extraText }: AppProps) {
7   return (
8     <>
9       <h1>{headerText}</h1>
10      {extraText && <p>{extraText}</p>}
11    </>
12  );
13 }
14

```

Default props - This is what you can set values as and will hold that value as default. the example bellow shows

```

6 export default function App({
7   headerText,
8   extraText = "default text",
9 }: AppProps) {

```

```

1  type ReadingCardProps = {
2    branchId: string;
3    minTempC: number;
4    maxTempC: number;
5    status: string;
6  };
7
8  export default function ReadingCard({
9    branchId,
10   minTempC,
11   maxTempC,
12   status,
13 }: ReadingCardProps) {

```

So this is basically defining the typing of the values that will be used in the rest of the code. It is being exported by its name ReadingCard.

 Questions / Queries / Pushback